
ODI Demo Builder — Session Guide

Andy and Niraj · Google Meet, each on their own machine · Jason is out sick — you are running this without him · ~3 hours ·
No skill required — paste the prompts below directly into Claude Code

How this works. This guide tells you exactly what to paste into Claude Code at each phase. Andy drives (runs commands, owns the Claude Code session). Niraj watches on the Meet and calls out content issues. Each phase has a prompt block with a red border — Andy copies it and pastes it in. Claude Code runs the session. Your job is to answer its questions and make the decisions marked below.

Before you start

5 minutes

ANDY — YOUR MACHINE

Confirm these are working before pasting anything:

```
claude whoami          # must print your email
op whoami              # must print your 1Password email
aws sts get-caller-identity --query Account --output text  # must print an AWS account ID
echo $FIVETRAN_API_KEY # must print a key (not blank)
echo $FIVETRAN_API_SECRET # must print a secret (not blank)
```

If any of these fail, fix it before proceeding. Paste the error in the Meet chat if you need help.

BOTH — DECIDE NOW ON THE MEET

Agree on three things before touching Claude Code:

1. **Industry vertical and fictional company name** — the company name must not be a real entity. If you are not sure, propose one and Claude Code will validate it.
2. **Buyer persona** — CDO, CIO, or Head of Data Engineering. This shapes the frontend copy.
3. **The 1Password item name for the SQL Server credentials** — Andy, what is the exact item name in your vault?

Phase 1 — Gather

target: 15 minutes

PHASE 1 — GATHER

ANDY DRIVES — open Claude Code and paste this prompt.

PASTE INTO CLAUDE CODE

You are running a Fivetran ODI demo build session for Andy and Niraj on Google Meet. Andy is an HVR veteran and SQL Server/CDC expert. Niraj sets demo content standards for the portfolio. The SE who normally facilitates (Jason Chletsos) is out sick. You are running the session directly.

Your role: facilitate a 3-hour build of a production-ready Fivetran ODI demo from scratch. Narrate

your reasoning at every decision point. Do not explain CDC to Andy – he knows it.

Collect these five inputs before anything else:

1. Industry vertical and fictional company name (validate the name is not a real entity before proceeding)
2. Buyer persona: CDO, CIO, or Head of Data Engineering
3. The 1Password item name for SQL Server credentials (you will fetch host, port, user, password using `op read`)
4. AWS region (default us-east-1) and S3 bucket name prefix
5. Target database and table selection (you will list these after connectivity is confirmed)

Ask for them now. Do not proceed until you have all five.

Claude Code will ask questions. Answer them. When it has all five it moves to connectivity testing automatically.

ANDY – CDC DECISION

Claude Code will ask you to choose the update method. It will show two options: BINARY_LOG_READER (CDC, log-based) and TELEPORT (full reload). Make the call as you would for a data-residency-sensitive enterprise customer. After you choose, Claude Code will walk through the Hybrid deployment framing — this is the 3-minute HVR replacement pitch. Let it run.

Phase 1 done when: connectivity confirmed, tables selected, update method chosen.

Phase 2 — Provision

target: 45 minutes

PHASE 2 — PROVISION

BOTH — DO THIS BEFORE PASTING THE PHASE 2 PROMPT

Go to fivetran.com → new destination → Managed Data Lake (MDLS). It shows you two values: a **Fivetran AWS account ID** and an **External ID**. Copy both. You need them for Terraform. Do not skip this — the IAM role will be wrong without them.

ANDY DRIVES — paste this when you have the two Fivetran values ready.

PASTE INTO CLAUDE CODE

Phase 1 is complete. Run Phase 2: Provision.

In order:

1. Scaffold the Terraform project at `/Users/jason.chletsos/Documents/GitHub/[DEMO_SLUG]-odi-demo/infra/`
2. Write `main.tf`: S3 bucket (versioning, AES256 encryption, public access block, 30-day lifecycle to STANDARD_IA), three Glue databases (bronze, silver, gold), IAM role for Fivetran MDLS with trust policy using `fivetran_aws_account_id` and `fivetran_external_id` variables, Athena workgroup, outputs for `bucket_name`, `fivetran_role_arn`, `bronze_db`, `silver_db`, `gold_db`.
3. Write `terraform.tfvars` with `aws_region`, `bucket_prefix`, `demo_slug`, and two placeholders for the Fivetran values. Pause and ask me to fill them in before applying.
4. After I confirm the values are entered, run `terraform init` and `terraform plan`. Print the plan. Wait for my explicit approval before running `apply`.

5. Run terraform apply after approval. Capture outputs into env vars.
6. Create the Fivetran MDLS destination via POST <https://api.fivetran.com/v1/destinations> using FIVETRAN_API_KEY and FIVETRAN_API_SECRET from the environment.
7. Create the SQL Server connector via POST <https://api.fivetran.com/v1/connectors> using 1Password credentials and the update method from Phase 1. Store the connector ID and deep link URL.
8. Test the connector: POST [/v1/connectors/\[ID\]/test](https://api.fivetran.com/v1/connectors/[ID]/test). Confirm setup_state is CONNECTED before proceeding.

Show me the Terraform plan before applying. Stop if the connector test fails – do not proceed to Phase 3 until it is CONNECTED.

NIRAJ – WATCH FOR

When Claude Code prints the Terraform plan, the S3 bucket name and Glue database names will be visible. Confirm the database names follow the pattern `[demo_slug]_bronze`, `[demo_slug]_silver`, `[demo_slug]_gold`. If the `demo_slug` contains the fictional company name, verify it does not collide with a real entity.

Phase 2 done when: Terraform applied, MDLS destination created, SQL Server connector CONNECTED.

If the connector test returns INCOMPLETE: Tell Claude Code the exact error text. Common cause with CDC: SQL Server Agent not running, or the user lacks VIEW SERVER STATE. Claude Code will give you the diagnostic SQL to run on the source server.

Phase 3 – Build

target: 60 minutes

PHASE 3 – BUILD

ANDY DRIVES — paste this prompt.

PASTE INTO CLAUDE CODE

Phase 2 is complete. Run Phase 3: Build.

In order:

1. Create the dbt project structure in `[DEMO_DIR]/dbt/`: directories `models/bronze`, `models/silver`, `models/gold`, `seeds`, `macros`, `tests`.
2. Write `dbt_project.yml`: project name from demo slug, Athena profile, silver as views, gold as Iceberg tables (parquet, partitioned), `store_failures` enabled.
3. Write `packages.yml` with `dbt_utils` and `dbt_expectations`.
4. Write `profiles.yml` using env vars for all connection details – nothing hardcoded. Target `awsdatacatalog` as the database value.
5. Generate `sources.yml` in `models/bronze/` for every table selected in Phase 1. Each table gets a `_fivetran_synced` not_null test and a `_fivetran_deleted` column documented.
6. For each source table: ask me for the primary key column, date columns, and amount/metric columns, then generate a staging model in `models/silver/` using source → renamed CTE pattern.
7. Ask me to identify the main dimension entity and main fact/event grain, then generate at least one gold dim model and one gold fct model using Iceberg materialization (`table_type='iceberg'`, `format='parquet'`, `partitioned_by`).
8. Apply Niraj's three content rules to everything generated:
 - No raw table or schema names in any string that renders in the UI

- Fictional org name used consistently in all copy
 - pipeline.json must include the connector ID and full deep_link URL for every connector
9. Generate the React frontend shell: App.tsx (HashRouter), Landing page with org name and buyer-persona copy, Pipeline page with connector cards that include working Open-in-Fivetran links, Analysis page with a Recharts chart placeholder.
 10. Run: `dbt deps && dbt seed && dbt build --select bronze silver gold`
 11. Report all results. If any model fails, diagnose from the error text before suggesting a fix.

Ask me about primary keys and grain before writing any SQL. Show me the model file list before running the build. Wait for my approval at both checkpoints.

NIRAJ — TWO APPROVAL CHECKPOINTS

Checkpoint 1: After Claude Code generates the sources.yml and before it writes silver models, it will ask about column structure. Tell Andy if the column naming needs adjusting for the demo narrative.

Checkpoint 2: After Claude Code applies the three content rules, review the output. The things to verify: no raw schema or table names in any UI string, the fictional org name is consistent, and the pipeline.json connector entry has a `deep_link` field with a real fivetran.com URL.

Phase 3 done when: dbt build passes, React shell generated, Niraj has reviewed the content.

If dbt build fails: Paste the full error into Claude Code. Do not try to fix it manually first — let Claude Code diagnose. Most common issues: `database: awsdatalog` is wrong in profiles.yml; Athena workgroup name mismatch; IAM permissions gap on the Glue catalog.

Phase 4 — Run

target: 30 minutes

PHASE 4 — RUN

ANDY DRIVES — paste this prompt.

PASTE INTO CLAUDE CODE

Phase 3 is complete. Run Phase 4: Run.

In order:

1. Trigger the first Fivetran sync: `POST /v1/connectors/[CONNECTOR_ID]/sync`
2. Poll sync status every 30 seconds and print each update: `GET /v1/connectors/[CONNECTOR_ID] → .data.status.sync_state`
3. When sync_state is SYNCED, verify Iceberg data: list S3 objects in the bronze prefix for metadata.json files, then run Athena `COUNT(*)` on each source table and print row counts.
4. If sync_state is ERROR, print the Fivetran error message and stop. Do not run dbt on errored data.
5. Run `dbt build --select silver gold` against the synced data.
6. Walk through the demo tour:
 - a. Landing page: org name visible, no raw table names, correct buyer persona copy
 - b. Pipeline page: SQL Server connector card visible, Open-in-Fivetran link opens the right connector
 - c. Fivetran dashboard at connector deep link: SYNCED status visible

```
d. Athena: SELECT * LIMIT 5 on each gold table, confirm rows
e. Analysis page: chart component renders
7. Print the final build summary with: demo slug, Fivetran group ID, connector ID, connector deep link, S3 bucket, Glue database names, dbt project name, update method, sync status, count of gold models built.
8. Print next steps: push repo to fivetran-jasonchletsos GitHub org, add to demo catalog, run /demo-blitz for full Niraj content audit and frontend-design pass, add analytical charts once data volume warrants.
```

Poll every 30 seconds. The first historical sync may take 5–30 minutes – this is expected.

BOTH — while the sync runs, use the time to review the dbt model files and React frontend structure that Claude Code generated in Phase 3. The sync polling will print updates automatically.

Phase 4 done when: sync SYNCED, Athena row counts confirmed, dbt gold models populated, demo tour complete, build summary printed.

Troubleshooting

- **op read returns empty:** Run `op signin` in a terminal tab, then tell Claude Code "1Password is signed in now, retry the credentials step."
- **SQL Server not reachable:** Check VPN first. If the server is in a private subnet, tell Claude Code — it will explain the Hybrid deployment option, which is the real-world customer answer to this situation.
- **Terraform IAM role fails:** The `fivetran_aws_account_id` in tfvars must match exactly what the Fivetran MDLS destination setup screen shows. Update tfvars and re-run.
- **Fivetran API 401:** `FIVETRAN_API_KEY` and `FIVETRAN_API_SECRET` must be set as separate env vars. Run `echo $FIVETRAN_API_KEY` — if blank, export them and retry.
- **dbt "database not found":** The `database:` in profiles.yml must be `awsdatacatalog`, not the Glue database name. The Glue database name goes in the `schema` field of sources.yml.
- **CDC sync stuck in SYNCING:** SQL Server Agent must be running and the sync user needs VIEW SERVER STATE permission. Tell Claude Code — it will give you the exact SQL to verify both.

Hybrid deployment talking points

Claude Code will surface these during Phase 2. Andy, this is the HVR replacement pitch for your banking and healthcare customers.

The objection: "We cannot send patient data / transaction records / PII to a third-party cloud."

The answer: Fivetran Hybrid Deployment runs the data-movement agent inside the customer's own network. Fivetran's cloud plane receives only orchestration signals and metadata — row counts, schema, sync timestamps. No customer data transits Fivetran's cloud infrastructure.

What stays identical: The connector config you set up today. The MDLS destination. The dbt pipeline. The React frontend. Hybrid deployment is a deployment mode, not a different architecture.

The pitch: "This is the HVR story with a managed control plane. You get log-based CDC without standing up and operating HVR yourself. Data stays in your network. Fivetran handles scheduling, schema drift detection, and delivery to your Iceberg lake."